

Lecture 3: Software Requirements Engineering

DM571 DM889: Software Engineering

Francesco Daghero

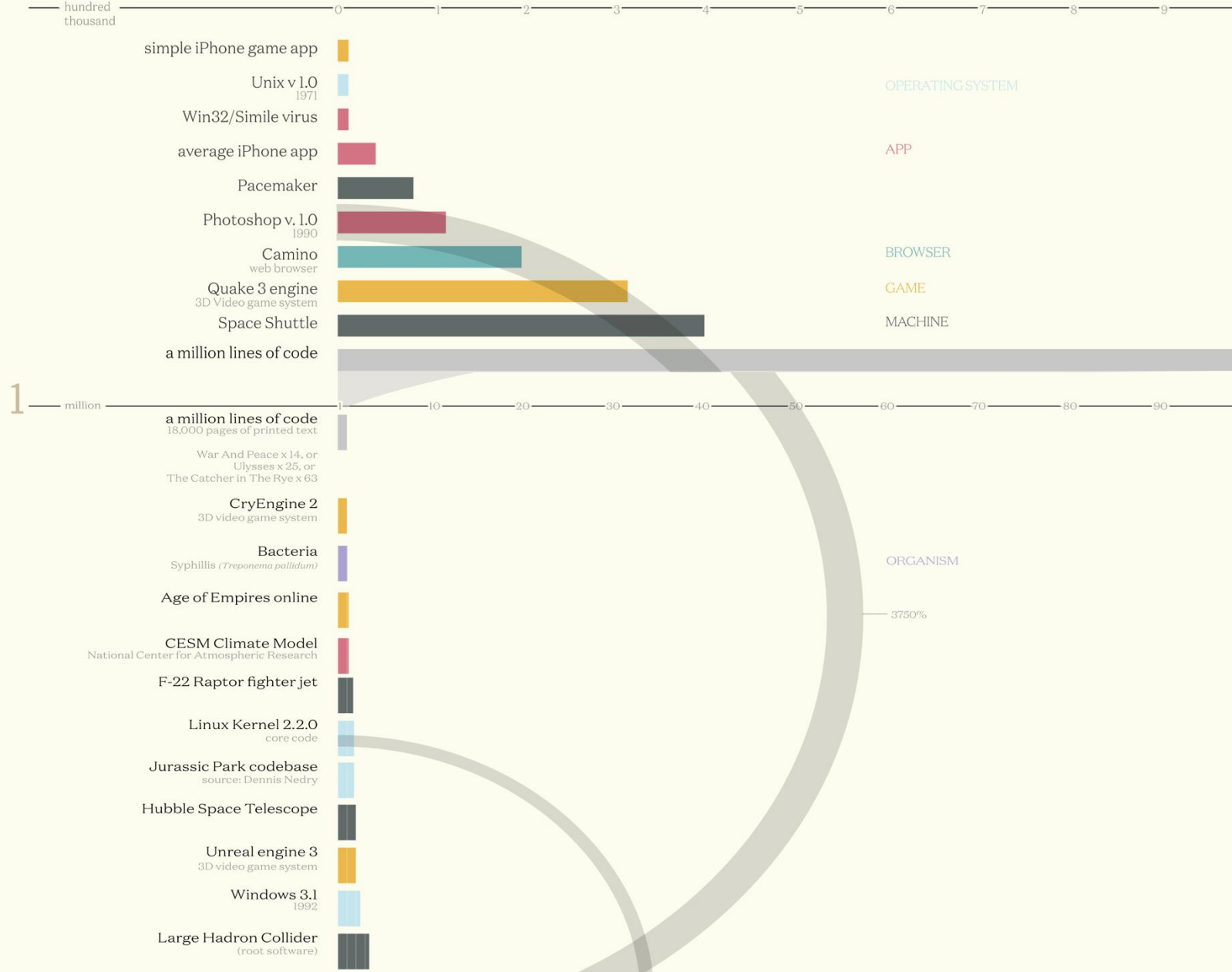
fdag@mmmi.sdu.dk

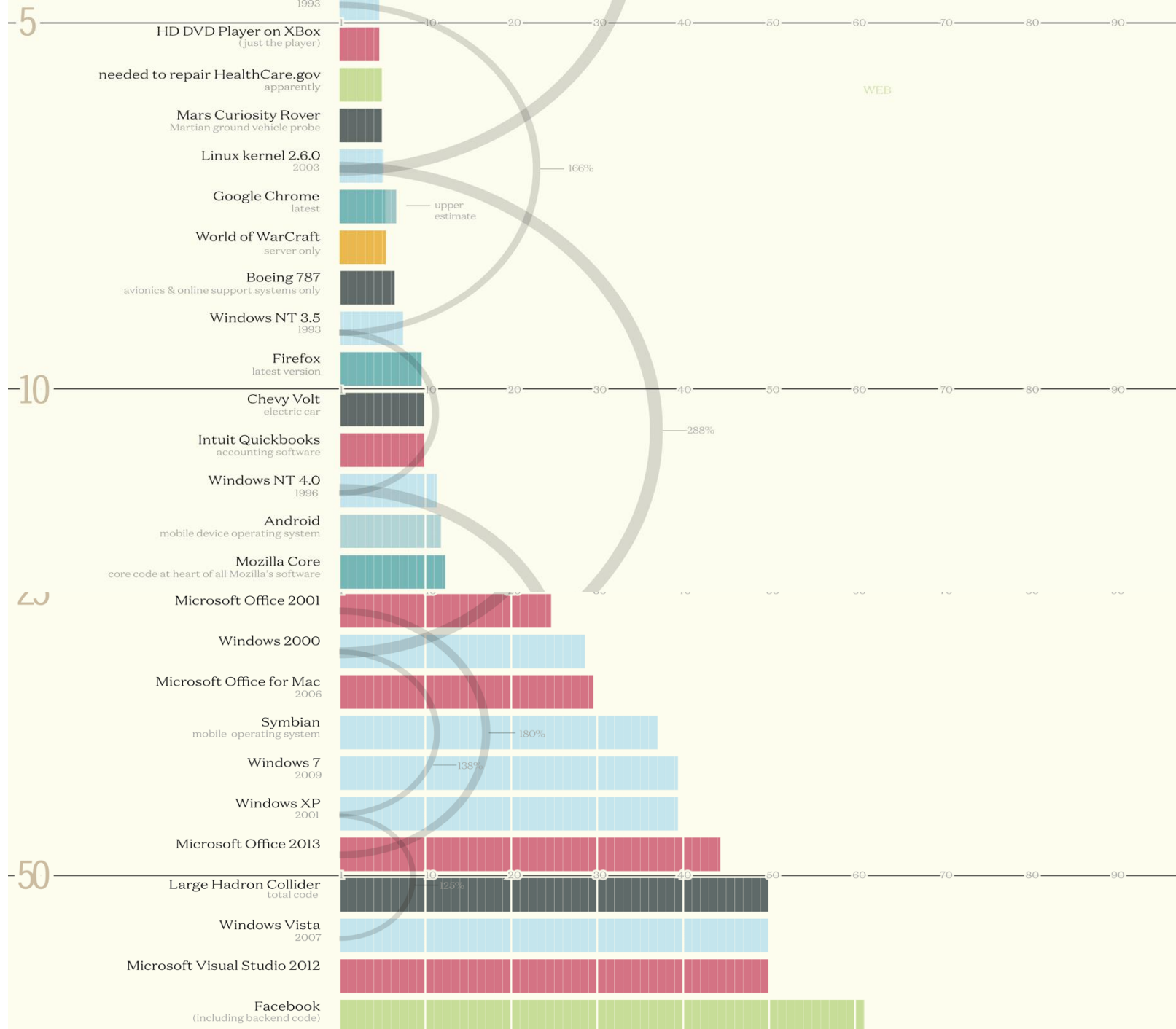
Tiziano Santilli

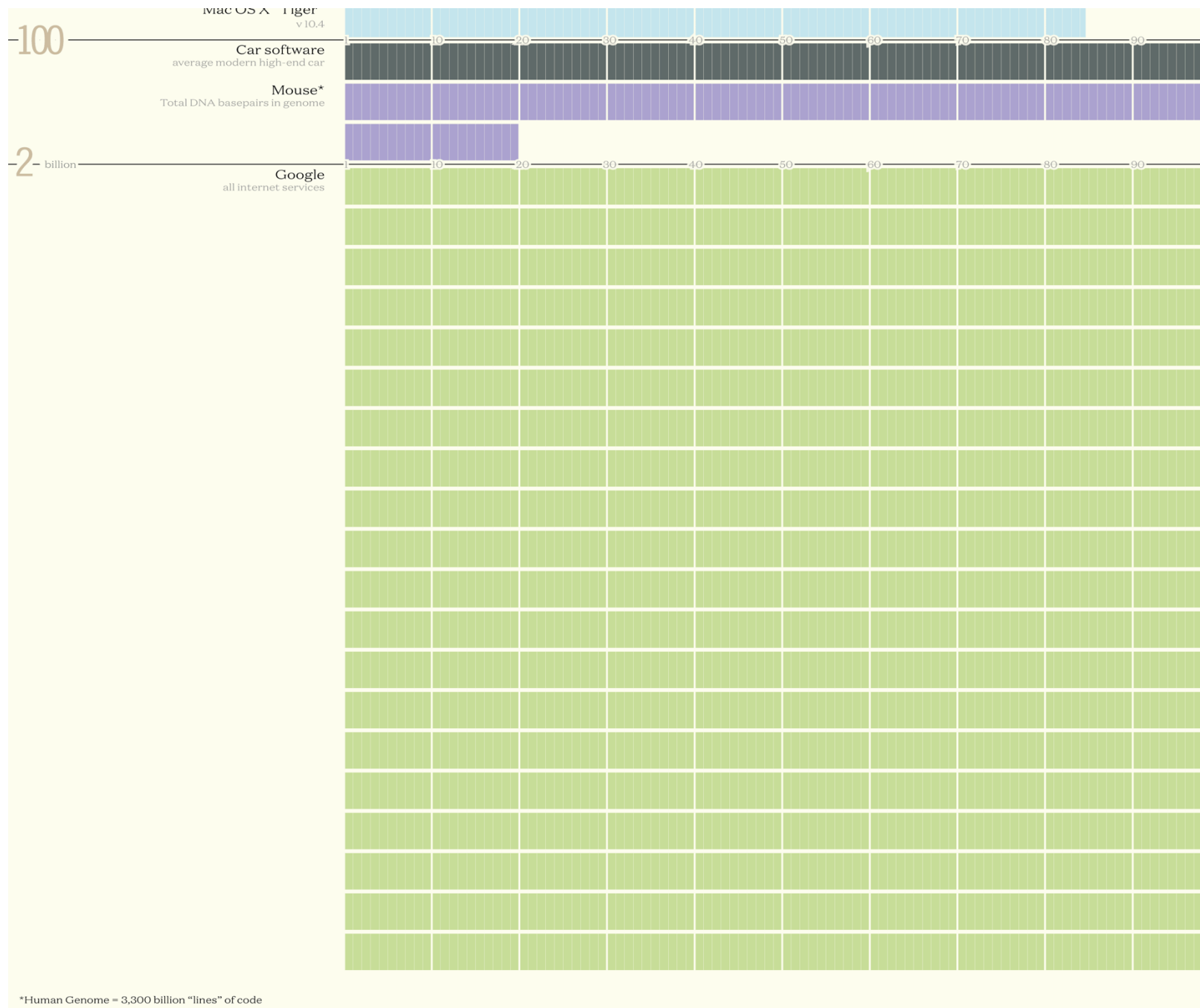
tisa@mmmi.sdu.dk

Codebases

Millions of lines of code







Agenda:

- Introduction
- Requirements
- Stakeholder
- Prioritization

Introduction

Introduction:

“I believe the hard part of building software is the specification, design, and testing of this conceptual construct, not the labour of representing it and test the fidelity of the representation.”

Fred Brooks

- Software has unique properties that differentiate it from other engineering artifacts.

Some of these properties are:

- Complexity
- Compliance
- Evolvability

Complexity:

- ✓ Software entities are more complex for their size than perhaps any other human construct.
- ✓ The complexity arises from the large number of unique interacting parts in a software system.
- ✓ As the number of entities increases, the number of interactions between them increases exponentially, making it impossible to know and understand all of them.
- ✓ Higher levels of software complexity increase the risk of unintentionally interfering with interactions, thus increasing the risk of introducing defects when changing the software.

Compliance:

- ✓ Conformity in software is the degree of adherence to preset expectations.
- ✓ The degree to which a product meets one or more prespecified criteria is called conformance in software engineering.
- ✓ There is no natural law underlying software!

Evolvability:

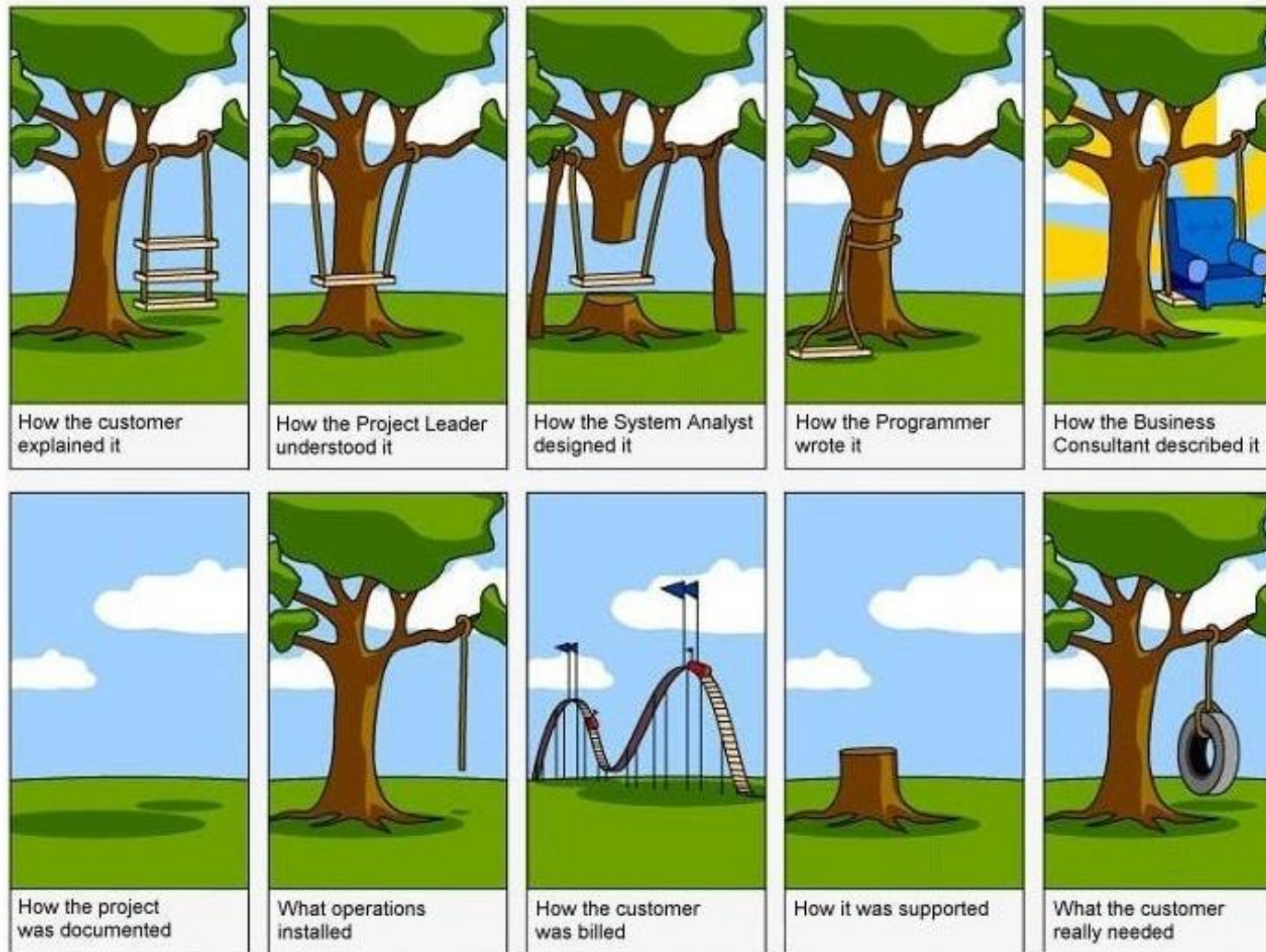
- ✓ Changeability in software is determined by the system's ability to change from one state to another to overcome encountered changes and deliver and provide for extended value.
- ✓ We are trying to hit the moving target when building software.
- ✓ If you are building something for a client, the client usually doesn't know in detail what they need until they see it.
- ✓ If you are working on a product, you never know in advance how the market will react to a release or a feature.

Introduction:

- Software processes refer to the methods and techniques used to develop and maintain software.
- There are several different software processes, but (almost) all involve:
 1. **Software Specifications:** The functionality of the software and constraints on its operation must be defined.
 2. **Software Development:** The software to meet the requirement must be produced.
 3. **Software Validation:** The software must be validated to ensure that it does what the customer wants.
 4. **Software Evolution:** The software must evolve to meet changing client needs.

Requirements

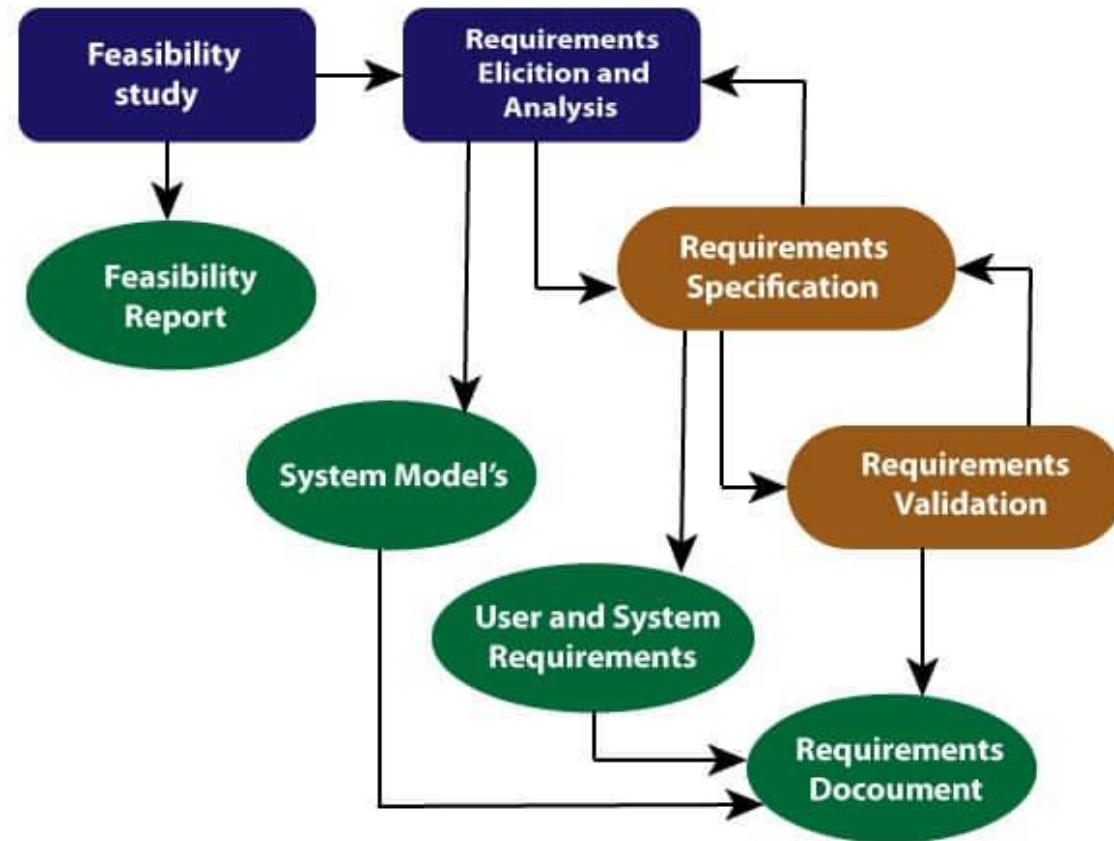
Introduction:



The ugly fact about requirements engineering. (from www.projectcartoon.com)

Introduction:

- In software engineering, requirements engineering is a systematic and disciplined process that involves several tasks to ensure the effective creation of a software product.
- The system requirements are the descriptions of the system services and constraints that are generated during the requirements engineering process.



Requirement Engineering Process

What is a requirement?

A requirement is a **feature** of new software that someone either **wants**, **needs**, or **commands**. It describes what the software does and any limitations it should have.

A requirement can be defined as follows:

1. A condition or capability needed by a **user** to solve a problem or achieve an objective.
2. A condition or capability that must be met or possessed by a **system** or system component to satisfy a contract, standard, specification, or other formally imposed documents.
3. A documented representation of a condition or capability as in 1 and 2.

Requirement Engineering Goal:

The goal of requirements engineering is to understand, record, and manage the needs and expectations of stakeholders (people related to the project) for a software system.

It helps in defining, documenting, and maintaining requirements in the engineering design process.

Requirement Engineering Steps:

1. **Feasibility Study:** This step focuses on analyzing the technical, operational, and economic feasibility of the project.
 - ❑ **Technical Feasibility:** Assess whether the current resources and technologies are sufficient to develop the project.
 - ❑ **Operational Feasibility:** Evaluate how easy the product will be to operate and maintain after deployment.
 - ❑ **Economic Feasibility:** Analysis of the cost and benefit of the project.
2. **Requirements Elicitation:** This involves gathering knowledge about the project domain and requirements from various sources, including customers, business manuals, existing software of the same type, standards, and other stakeholders.
3. **Requirements Specification:** This step involves writing the requirements in technical language so that they can be understood and used by the development team.
4. **Requirements Verification and Validation:** This ensures that the specified requirements are correct and meet the needs and expectations of the stakeholders.
5. **Requirements Management:** This involves managing the requirements as they are transformed into a working system.

Guided Exercise

Project overview:

You have been hired as a Requirements Engineer at a software development company. The client is a hospital that needs a software system to manage patient medical records.

The system must allow two types of users to access the records:

A Doctor, who needs to manage and update patient information

A Patient, who needs to access their own medical record

Your task: Write all the requirements and functionalities for this software. These will later be handed off to the development team.

Requirements Properties:

1. **Validity:** A requirement is valid if it represents a real need, expectation, or requirement of the stakeholder. It should be based on accurate information and should contribute to the objectives of the system.
2. **Consistency:** Consistency refers to the absence of conflict or contradiction among the requirements. All requirements should be in harmony with each other and with the system as a whole.
3. **Completeness:** Completeness means that all necessary parts of the requirement have been included and fully described. A complete requirement leaves no room for ambiguity or misinterpretation.
4. **Realism:** Realism refers to the feasibility of the requirement. The requirement should be achievable within the constraints of the system, such as time, cost, technology, and resources.
5. **Verifiability (Certifiability):** A requirement is verifiable if there exists some finite cost-effective process with a known level of confidence that can check whether the system meets the requirement. This often involves testing or inspection.

- ✓ These properties help ensure that the system or product developed meets the user's needs and expectations, and they guide the development process to prevent miscommunication and misunderstandings.

Type of Requirement:

User Requirements:

- User requirements describe what the user does with the system, such as what activities that users must be able to perform.

System Requirements:

- System requirements describe what the system should do to satisfy user requirements.

Requirements are typically divided into **functional** and **non-functional** requirements.

Functional and Non-Functional Requirements:

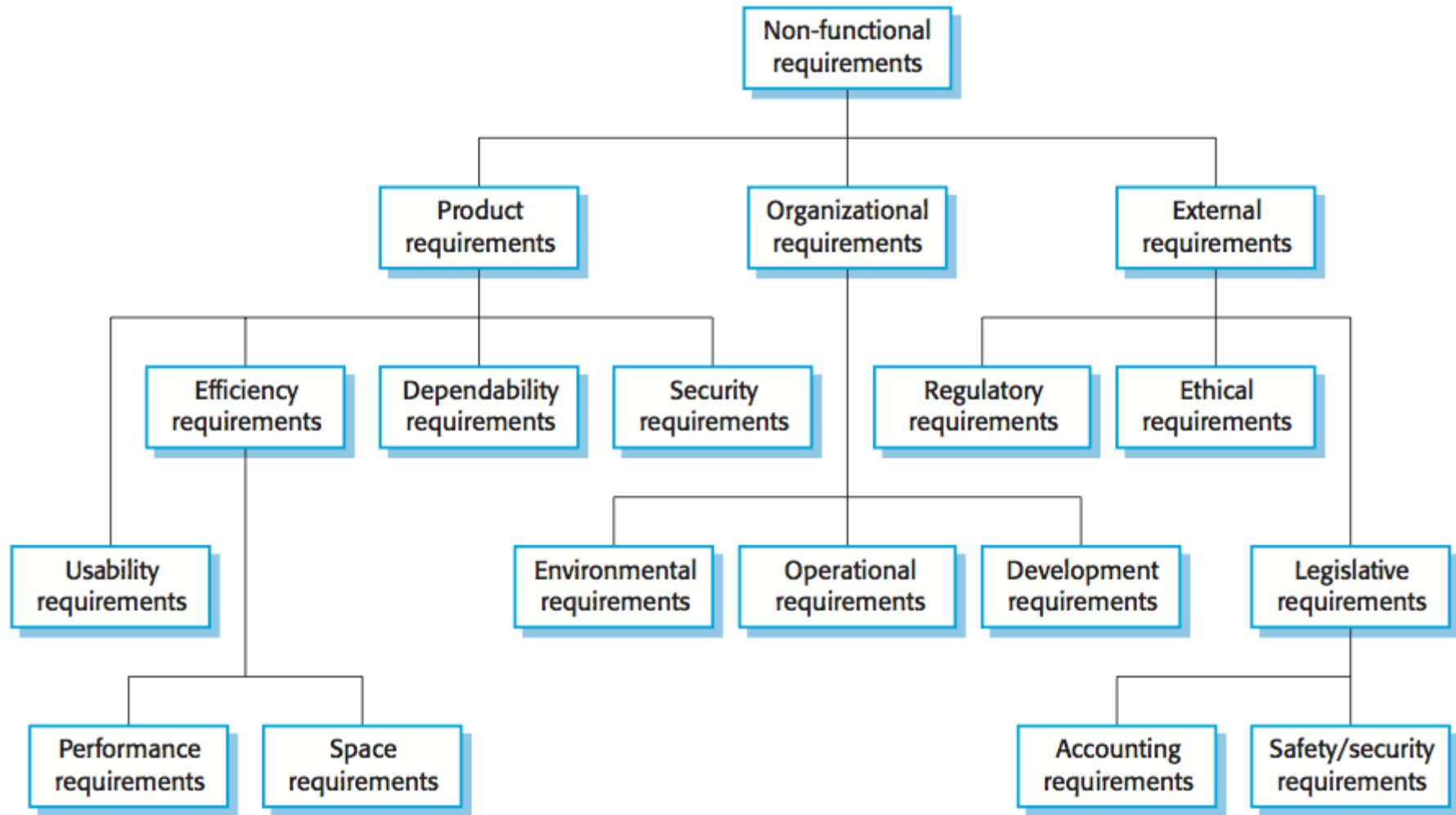
Functional requirements:

- Statements of the services the system should provide, how it should react to particular inputs, and how it should behave in particular situations.
- May state what the system should not do.

Non-functional requirements:

- Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc.
- Often apply to the system as a whole rather than individual features or services.

Types of Non-Functional Requirement:



Reminder:

Remember, the key to a successful project is a clear and thorough understanding of both the functional and non-functional requirements. These requirements should be documented, reviewed, and agreed upon by all **stakeholders**.

Examples:

Functional Requirements: Functional requirements define the basic system behavior. Essentially, they are what the system does or must not do, and can be thought of in terms of how the system responds to inputs. Functional requirements usually have more detailed specifications since they describe the behavior of the system making them easier to infer from requirements.

1. **User Authentication:** The system should authenticate users before granting access to the application.
2. **Search Functionality:** Users should be able to search for products using keywords.
3. **Payment Processing:** The system should allow users to make payments through credit/debit cards, PayPal, etc.

Non-Functional Requirements: Non-functional requirements, on the other hand, define how the system should behave; they place constraints on the system's functionality. Non-functional requirements are often referred to as the “quality attributes” of a system.

1. **Performance:** The system should handle 10,000 concurrent users, and response time should not exceed 5 seconds.
2. **Security:** All sensitive data should be encrypted, and no personal user data should be stored on the device.
3. **Usability:** The system should be easy to use, and should not require the user to read a manual.

Functional requirement (Example):

Building a Personal Blog Website.

- ❑ **User Registration and Authentication:** Users should be able to register and create an account using their email address. They should be able to authenticate themselves using their username and password.
- ❑ **Content Creation:** The blogger should be able to create, edit, and delete blog posts. This includes adding text, images, and multimedia content to the posts.
- ❑ **Commenting:** Readers should be able to leave comments on blog posts. They should also be able to reply to other comments.
- ❑ **Search Functionality:** Users should be able to search for blog posts using keywords.
- ❑ **Category/Tagging System:** The blogger should be able to categorize or tag posts. Users should be able to view posts by category or tag.
- ❑ **RSS Feed:** The blog should have an RSS feed that users can subscribe to.
- ❑ **Social Media Integration:** Users should be able to share blog posts on various social media platforms.

Non-Functional requirement (Example):

Building a Personal Blog Website.

- ❑ **Performance:** The website should load in less than 2 seconds when accessed with a standard broadband connection. The system should be able to handle up to 1000 simultaneous users.
- ❑ **Availability:** The website should be available 24/7, with a downtime of less than 0.1%.
- ❑ **Usability:** The website should be easy to use, with a clean and intuitive interface. The website should be navigable with a keyboard only.
- ❑ **Security:** User data should be stored securely. Passwords should be hashed and not stored in plain text. The website should use HTTPS for secure communication.
- ❑ **Scalability:** The website should be able to handle an increase in users and data volume without a significant impact on performance.
- ❑ **SEO Optimization:** The website should be optimized for search engines, with appropriate use of meta tags, alt tags, and structured data.
- ❑ **Accessibility:** The website should comply with WCAG 2.1 Level AA accessibility standards, ensuring it is accessible to people with a range of disabilities.
- ❑ **Cross-Browser Compatibility:** The website should function correctly in the latest versions of major browsers such as Chrome, Firefox, Safari, and Edge.

User Requirement (Example):

Software System: Food Delivery App

Functional Requirements

1. User Registration and Authentication

- The system should allow new users to register by providing necessary details like name, email, and contact number.
- The system should authenticate users before they can access their accounts.

2. Restaurant Listing

- The system should provide a list of restaurants based on the user's location.
- Each restaurant listing should include details like menu, prices, ratings, and estimated delivery time.

3. Order Placement

- Users should be able to select items from a restaurant's menu and add them to a cart.
- Users should be able to place an order by specifying a delivery address and making a payment.

4. Order Tracking

- The system should provide real-time updates on the status of the user's order.
- Users should be able to track the delivery on a map.

5. Ratings and Reviews

- Users should be able to rate and review restaurants and their food items.

User Requirement (Example):

Software System: Food Delivery App

Non-Functional Requirements

1. Usability

- The app should have an intuitive and user-friendly interface.

2. Performance

- The app should load restaurant listings and track orders in real time.

3. Security

- User data should be stored securely. All transactions should be encrypted.

4. Scalability

- The system should be able to handle a large number of users and orders simultaneously.

Guided Exercise 2

Project overview:

You have been hired as a Requirements Engineer at a software development company. The client is a hospital that needs a software system to manage patient medical records.

The system must allow two types of users to access the records:

A Doctor, who needs to manage and update patient information

A Patient, who needs to access their own medical record

Your task: Write all the requirements and functionalities for this software. These will later be handed off to the development team. **Divide the requirements in Functional and Non-functional requirements**

Gathering Requirements Steps:

- Identifying stakeholders
- Establishing goals and objectives
- Eliciting requirements from the stakeholders
- Documenting the requirements
- Analyzing and confirming the requirements
- Prioritizing

Stakeholders

Stakeholder:

- Any person or organization who is affected by the system in some way and so who has a legitimate interest.

Stakeholder types

- End users
- System managers
- System owners
- External stakeholders

Building a Personal Blog Website.

1. Website Owner/Blogger
2. Web Developer/Designer
3. Hosting Provider
4. Readers/Visitors
5. Advertisers
6. SEO Specialist
7. Content Creator

- ✓ It is important to identify all stakeholders, understand their needs and expectations, and manage their influence in relation to the project requirements to ensure a successful project outcome.

Identifying Stakeholder:

Key personnel:

- Decision-makers
- End-users
- System administrators
- Engineering
- Marketing
- Sales
- Customer supports

Guided Exercise 3

Project overview:

You have been hired as a Requirements Engineer at a software development company. The client is a hospital that needs a software system to manage patient medical records.

The system must allow two types of users to access the records:

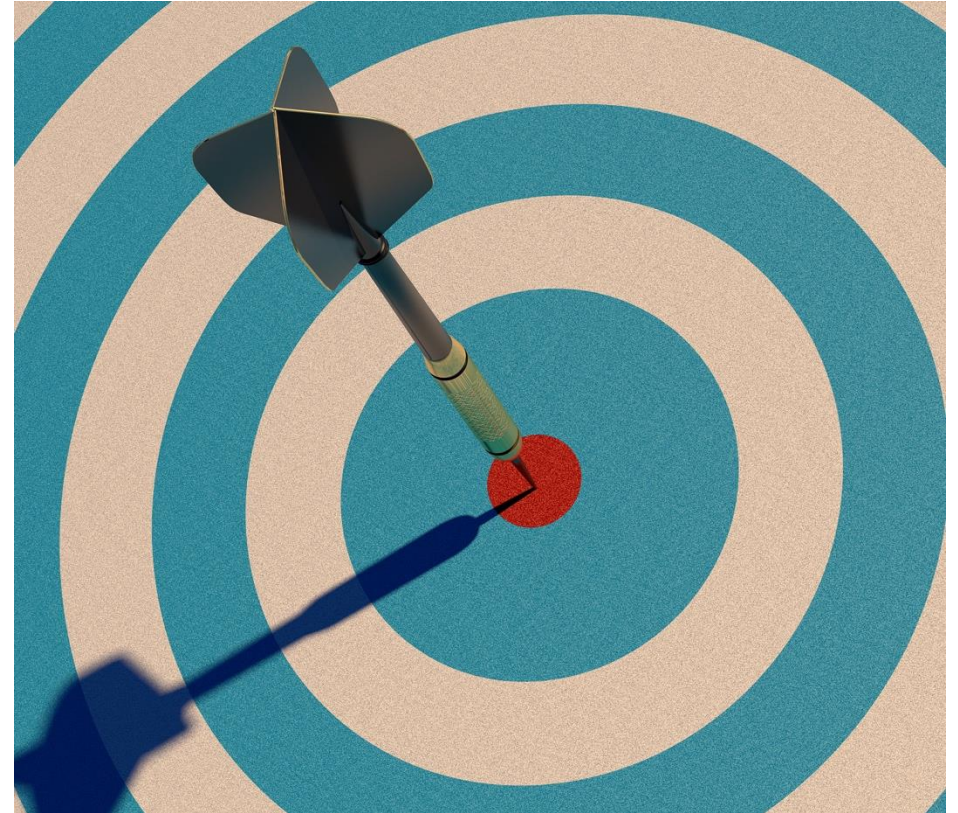
A Doctor, who needs to manage and update patient information

A Patient, who needs to access their own medical record

Your task: Identify the Stakeholders

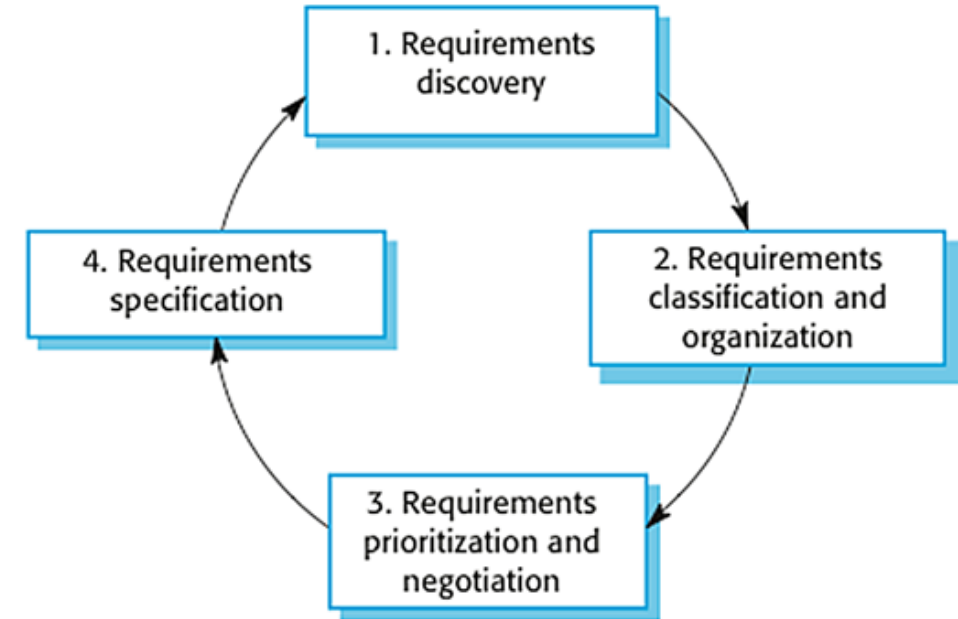
Establishing Goals and Objectives:

- **Goals:** broad, long-term achievable outcomes
Goals can include customer outcomes and business goals.
- **Objectives:** actionable, measurable actions that achieve the goal



Eliciting Requirements from Stakeholders:

- **Interviews:** Direct interaction with stakeholders to understand their needs.
- **Surveys/Questionnaires:** Collecting data from a large group of users.
- **Observation:** Understanding user behavior and needs in their natural environment.
- **Document Analysis:** Reviewing existing documentation related to current systems and processes.



Problems of Requirements Elicitation:

- Stakeholders don't know what they really want.
- Stakeholders express requirements in their own terms.
- Different stakeholders may have conflicting requirements.
- Organisational and political factors may influence the system requirements.
- The requirements change during the analysis process. New stakeholders may emerge and the business environment may change.

Ways of Documenting Requirements:

Notation	Description
Natural language	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language like a programming language, but with more abstract features to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract

Analyzing and Confirming the Requirements:

In this step we ensure that we are building the right features for the right reasons before committing development resources.

We need to analyze the requirements to ensure their **consistency**, **clarity**, and **completeness**.

Confirmation of requirements is sharing and approving the requirements with the stakeholders.

- ☐ **Prototyping:** Build simplified versions to get early feedback
- ☐ **User Interviews:** Directly confirm requirements with end-users
- ☐ **A/B Testing:** Test concepts with small user segments
- ☐ **Feasibility Analysis:** Confirm technical and resource viability
- ☐ **Acceptance Criteria:** Define clear success metrics for each requirement

Prioritization

Prioritizing:

Considering limited resources and competing demands, the ability to effectively prioritize software requirements is what separates successful projects from failed ones.

Why Prioritize Requirements?

- **Limited Resources:** Time, budget, and team capacity constraints
- **Business Value:** Focus on features that deliver the highest return on investment (ROI)
- **Risk Management:** Address high-risk items appropriately
- **Stakeholder Alignment:** Create consensus on what matters most
- **Technical Dependencies:** Ensure logical development sequence

Trivial Prioritization Techniques:

Numerical Assignment: A straightforward approach where requirements are assigned numerical priority values or simple classification labels.

- Easy to implement, universally understood, and requires minimal training, but may lead to priority inflation and subjective assessments.

Weighted Scoring

A multi-dimensional evaluation method that assesses requirements across various business and technical criteria with different importance weights.

- Provides nuanced analysis, reduces bias through multiple criteria, creates defensible decisions, and adapts to different organizational priorities.

Common Prioritizing Methods:

MoSCoW method:

MoSCoW categorizes requirements based on their criticality to project success and business objectives.

- **Must Have:** Critical requirements that must be delivered
 - **Should Have:** Important but not critical requirements
 - **Could Have:** Desirable requirements if resources permit
 - **Won't Have:** Requirements that won't be delivered in this iteration
- ✓ Easy to understand, facilitates quick decisions, creates clear categories, and works well for time-boxed projects.

Common Prioritizing Methods:

Kano Model:

Kano is a customer-centric framework that categorizes requirements based on how they influence user satisfaction and dissatisfaction.

- **Basic/Threshold Attributes:** Must-have features customers expect
 - **Performance Attributes:** Features where more is better
 - **Excitement Attributes:** Unexpected features that delight users
 - **Indifferent Attributes:** Features users don't care about
- ✓ Focuses on user experience, helps identify innovation opportunities, differentiates between expected and delightful features, and provides market advantage insights.

Common Prioritizing Methods:

ICE scoring:

ICE scoring prioritizes requirements using three set measurements: Impact, Confidence, and Ease.

A systematic framework that evaluates requirements based on three critical dimensions: Impact, Confidence, and Ease, creating a holistic assessment of priority.

- **Impact:** Potential effect on key metrics (1-10)
- **Confidence:** Certainty level about impact estimates (1-10)
- **Ease:** Implementation difficulty and resource requirements (1-10)

Calculate ICE score by multiplying Impact × Confidence × Ease (or averaging them)

- ✓ Balances multiple perspectives, accounts for uncertainty, scales well across project sizes, and **provides numerical ranking for clear decision-making.**

Guided Exercise 3

Project overview:

You have been hired as a Requirements Engineer at a software development company. The client is a hospital that needs a software system to manage patient medical records.

The system must allow two types of users to access the records:

A Doctor, who needs to manage and update patient information

A Patient, who needs to access their own medical record

Your task: Prioritize the requirements with MoSCoW and Kano